STREAMLIT

Handwritten Notes

From first script to deployed ML, DL & LLM apps

1. **Basics & execution model**
2. **Widgets, layout, forms, multipage**
3. **Project: Indian Railway Reservation System**
4. **ML & Deep Learning deployment**
5. **Chatbot with Ollama + Qwen**

Author: Chandan Chaudhari

GitHub: github.com/chandanc5525

Written for Streamlit 1.6x | Python 3.10+ | October 2026

Core code was run & tested with Streamlit AppTest - build along!

Learning roadmap



> **TIP:** How to use these notes: type every example yourself, run it with `streamlit run file.py`, then change one thing and watch what happens. Streamlit rewards experimenting.

Contents

1. Streamlit Basics 4

1.1 What is Streamlit? 4
1.2 Install and first app 4
1.3 The execution model (most important idea!) 5
1.4 Text elements 5
1.5 Showing data 6
1.6 Built-in charts 6
1.7 Images, audio, video 7
1.8 Caching - make apps fast 7
1.9 Session state in depth 8
1.10 Typical project structure 8

2. Widgets & Layout 9

2.1 Anatomy of a widget 9
2.2 Buttons 9
2.3 Text, number, date inputs 10
2.4 Choice widgets 10
2.5 File, camera and audio inputs 10
2.6 Editable tables - st.data_editor 11
2.7 Layout: sidebar, columns, tabs, containers 11
2.8 Status and feedback messages 12
2.9 Forms - batch the inputs 12
2.10 Callbacks, fragments and flow control 13
2.11 Multipage apps 13
2.12 Config, theme and secrets 14
2.13 Widget cheat sheet 14

3. Railway Reservation 16

3.1 Problem statement 16
3.2 Architecture and folders 16
3.3 Database design 17
3.4 Business rules 17
3.5 db.py - connection, schema, sample data 18
3.6 services.py - the business logic 19
3.7 app.py - the Streamlit interface 22
3.8 User journey 25
3.9 Testing the app 25
3.10 Ideas to extend the project 26

4. ML & DL Deployment 27

4.1 The deployment pattern 27
4.2 Saving and loading models 27
4.3 Example 1 - scikit-learn classifier 28
4.4 Good practices for ML apps 29
4.5 Example 2 - deep learning image classifier 30
4.6 Example 3 - train your own CNN (MNIST digits) 31
4.7 Performance and memory 32
4.8 Deploying: where and how 32
4.9 Common errors 33

5. Ollama + Qwen Chatbot 34

5.1 What are Ollama and Qwen? 34
5.2 Install and download a model 34
5.3 The Ollama Python client 35
5.4 Streamlit chat building blocks 35
5.5 The complete chatbot 36
5.6 Upgrades you can add 37
5.7 Troubleshooting 39
5.8 Deploying the chatbot 39
5.9 Project ideas 40

6. Revision Pages 41

6.1 Streamlit in one page 41
6.2 Mistakes beginners make 41
6.3 Interview questions 42
6.4 Practice tasks 42

1 Streamlit Basics

what it is, how it runs, text, data, charts, caching, state

1.1 What is Streamlit?

*Streamlit is an **open-source Python framework** that turns a plain Python script into an interactive web app. You write no HTML, CSS or JavaScript - every `st.something()` call draws a UI element, and widgets simply **return Python values**.*

- *Great for: dashboards, data exploration, ML/DL demos, internal tools, LLM chatbots.*
- *Runs a local web server (default port **8501**) and opens the browser for you.*
- *Everything is pure Python, so data scientists can ship apps without a front-end team.*
- *Not ideal for: huge multi-user public sites with complex custom UI or pixel-perfect design (use Django / FastAPI + React there).*

Tool	Best at	Style
Streamlit	Data apps, dashboards, chatbots, quick prototypes	Script reruns top-to-bottom
Gradio	Quick ML model demos (input -> output)	Function + interface
Dash (Plotly)	Analytical dashboards, fine control	Explicit callbacks
Flask / Django	Full websites, REST APIs, auth, ORM	Routes + templates

1.2 Install and first app

>> terminal

```
# 1) create an isolated environment (recommended)
python -m venv venv
venv\Scripts\activate            # Windows
source venv/bin/activate           # macOS / Linux

# 2) install and test
pip install streamlit
streamlit --version
streamlit hello                    # opens the built-in demo
```

>> hello.py

```
# hello.py
import streamlit as st

st.title("My first Streamlit app")
name = st.text_input("What is your name?")
if name:
    st.write(f"Namaste, **{name}**!")
```

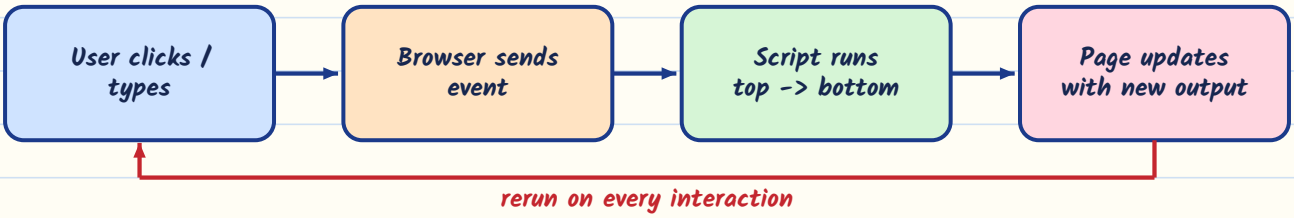
>> run it

```
streamlit run hello.py                       # http://localhost:8501
streamlit run hello.py --server.port 8080    # custom port
python -m streamlit run hello.py             # if 'streamlit' is not on PATH
```

> **TIP:** Save the file and Streamlit shows a **Rerun / Always rerun** button at the top-right - choose *Always rerun* for instant live-coding. Stop the server with Ctrl+C.

1.3 The execution model (most important idea!)

Every time the user touches a widget (or the file changes), Streamlit **re-runs your whole script from top to bottom**. A widget's function call returns its *current* value during that run. Ordinary variables are re-created on every run.



>> the classic bug

```
import streamlit as st

count = 0                        # reset to 0 on EVERY rerun!
if st.button("Add one"):
    count += 1
st.write("Count:", count)        # always shows 0 or 1 - never 2
```

>> fix: session_state remembers values

```
import streamlit as st

if "count" not in st.session_state:      # create once per browser session
    st.session_state.count = 0
if st.button("Add one"):
    st.session_state.count += 1
st.write("Count:", st.session_state.count)   # 1, 2, 3 ...
```

> **REMEMBER:** Three ways to keep things between reruns: **session_state** (per user session), **@st.cache_data / @st.cache_resource** (expensive work), and **external storage** (database, file) for permanent data.

1.4 Text elements

Call	Result
`st.title('Text')`	Biggest heading (use once per page)
`st.header / st.subheader`	Section headings (accept divider='rainbow')
`st.text('plain')`	Fixed-width plain text
`st.markdown('**bold** _italic_')`	Markdown, :red[colored text], :material/home: icons, emoji :tada:
`st.caption('small grey')`	Small helper / footnote text
`st.code(code, language='python')`	Syntax-highlighted code block with copy button

Call	Result
st.latex(r'\int_0^1 x^2 dx')	Math formulas using LaTeX
st.divider()	Horizontal line
st.write(anything)	Swiss-army knife: text, DataFrame, dict, chart, model ...

>> text_demo.py

```
import streamlit as st

st.title("Student Marks Report")
st.header("Class 12-A")
st.subheader("Mathematics")
st.markdown("Result: :green[**PASS**] and :red[2 absentees]")
st.caption("Data updated daily")
st.code("print('hello')", language="python")
st.latex(r"\sigma = \sqrt{\frac{1}{N}\sum (x_i-\mu)^2}")
st.divider()
```

> **NOTE: Magic:** a bare variable or expression on its own line (like `df` or `'hello'`) is displayed automatically - same as `st.write`.

1.5 Showing data

`st.dataframe` is interactive (sort, search, resize, download). `st.table` is static. `st.metric` shows a KPI with a delta arrow. `st.json` prints collapsible JSON.

>> data_demo.py

```
import pandas as pd, streamlit as st

df = pd.DataFrame({
    "City": ["Mumbai", "Delhi", "Chennai", "Kolkata"],
    "Population (M)": [20.4, 32.9, 11.5, 15.1],
    "Literacy %": [89.7, 86.3, 90.3, 87.1],
})
st.dataframe(df, width="stretch", hide_index=True,
             column_config={"Literacy %": st.column_config.ProgressColumn(
                 "Literacy %", min_value=0, max_value=100, format="%.1f")})

c1, c2, c3 = st.columns(3)
c1.metric("Total cities", len(df))
c2.metric("Avg literacy", f"{df['Literacy %'].mean():.1f}%", delta="+1.2%")
c3.metric("Max population", df["Population (M)"].max(), delta="-0.4", delta_color="inverse")
st.json({"state": "ok", "rows": len(df)})
```

> **CAREFUL:** `use_container_width=True` is **deprecated**. Use `width="stretch"` (fill the parent) or `width="content"` instead. The same applies to buttons, charts, images and data editors.

1.6 Built-in charts

Quick charts need only a DataFrame: `st.line_chart`, `st.area_chart`, `st.bar_chart`, `st.scatter_chart`, and `st.map` (needs `lat`/`lon` columns). For full control use Matplotlib (`st.pyplot`), Plotly (`st.plotly_chart`) or Altair (`st.altair_chart`).

>> chart_demo.py

```
import numpy as np, pandas as pd, streamlit as st

data = pd.DataFrame(np.random.randn(30, 3).cumsum(axis=0), columns=["A", "B", "C"])
st.line_chart(data)
st.bar_chart(data.tail(5), horizontal=True)
st.area_chart(data, color=["#ff4b4b", "#1c83e1", "#21c354"])
st.scatter_chart(data, x="A", y="B", size="C")

# Plotly example
import plotly.express as px
fig = px.pie(values=[40, 35, 25], names=["SL", "3A", "2A"], title="Class mix")
st.plotly_chart(fig, width="stretch")

# map
st.map(pd.DataFrame({"lat": [19.07, 28.61], "lon": [72.87, 77.20]}))
```

1.7 Images, audio, video

>> media

```
st.image("photo.jpg", caption="Gateway of India", width=300)
st.audio("song.mp3")
st.video("lecture.mp4")            # file path or YouTube/URL
st.image(["a.png", "b.png"], width=150)   # several images side by side
```

1.8 Caching - make apps fast

Since the script reruns constantly, slow work (reading a big CSV, calling an API, loading a model) must be cached.

	@st.cache_data	@st.cache_resource
Use for	Data you compute: DataFrames, lists, numbers, API results	Objects you share: ML model, DB connection, tokenizer
Returns	A fresh **copy** each call (safe to mutate)	The **same object** for everyone (not copied)
Needs to be	Pickle-able	Anything (even un-pickle-able)
Options	ttl=600, max_entries, show_spinner	ttl, validate=function, show_spinner
Clear	func.clear() / st.cache_data.clear()	st.cache_resource.clear()

>> caching_demo.py

```
import time, pandas as pd, streamlit as st

@st.cache_data(ttl=600, show_spinner="Reading data...")   # refresh every 10 min
def load_csv(path):
    time.sleep(3)                       # pretend this is slow
    return pd.read_csv(path)

@st.cache_resource
def get_model():
    import joblib
    return joblib.load("model.joblib")  # loaded ONCE, shared by all users

df = load_csv("sales.csv")              # slow first time, instant afterwards
model = get_model()
```

> **CAREFUL:** `@st.cache` was removed long ago - never use it in new code. Cache keys are built from the function's **arguments**, so same arguments = cached result.

1.9 Session state in depth

- `st.session_state` is a dict-like object that lives for one browser tab session.
- Give a widget a `key=` and its value is stored automatically at `st.session_state[key]`.
- Use **callbacks** (`on_click`, `on_change`) - they run *before* the rerun, so state is already updated.
- A different tab / refresh (F5) starts a **new** session with empty state.

>> session_demo.py

```
import streamlit as st

st.session_state.setdefault("cart", [])         # initialise once

def add_item():
    st.session_state.cart.append(st.session_state.item)   # read widget by key
    st.session_state.item = ""                            # clear the box

st.text_input("Item", key="item")
st.button("Add", on_click=add_item)
st.write("Cart:", st.session_state.cart)
st.button("Empty cart", on_click=lambda: st.session_state.cart.clear())
```

> **CAREFUL:** You cannot assign to `st.session_state[key]` of a widget **after** that widget is drawn in the same run - set it before the widget or inside a callback.

1.10 Typical project structure

>> folders

```
my_app/
|-- app.py                # entry point (streamlit run app.py)
|-- pages/                # optional multipage files
|-- utils/ or services.py # plain Python logic (no st.* inside)
|-- data/                 # csv, sqlite, models
|-- .streamlit/
|     |-- config.toml     # theme + server settings
|     |-- secrets.toml    # API keys (never commit to Git!)
|-- requirements.txt      # pinned dependencies
|-- README.md
```

> **REMEMBER:** Golden rule: keep **business logic** in normal Python modules and keep **UI code** in the Streamlit file. You get easier testing and the app stays readable.

2 Widgets & Layout

inputs, choices, uploads, layout, forms, multipage apps, config

2.1 Anatomy of a widget

*Every widget is a function call that **draws a control and returns its current value**. Common parameters work on almost all widgets:*

Parameter	Meaning
`label`	*Text shown to the user (required, first argument)*
`key`	*Unique id; value lives in st.session_state[key]*
`help`	*Tooltip text shown on a small (?) icon*
`disabled`	*True greys the widget out*
`label_visibility`	*'visible' | 'hidden' | 'collapsed'*
`on_change / on_click`	*Callback function run BEFORE the next rerun*
`args / kwargs`	*Arguments passed to the callback*
`width`	*'stretch' | 'content' | pixels (newer versions)*

> ***CAREFUL:*** *Two widgets with the same label and parameters get the same internal id - Streamlit raises* `DuplicateWidgetID`*. Fix it by giving each a unique* `key=`*. This is the #1 error inside loops!*

2.2 Buttons

>> buttons.py

```
clicked = st.button("Submit", type="primary", help="Send the form")   # True ONLY on the click rerun
st.download_button("Download CSV", data=df.to_csv(index=False),
                   file_name="data.csv", mime="text/csv")
st.link_button("Open Streamlit docs", "https://docs.streamlit.io")

if st.button("Say hi"):
    st.write("Hi!")            # disappears on the next interaction!
```

> ***REMEMBER:*** *A button is* `True` *for **one rerun only**. To keep something shown after the click, store a flag in* `session_state`*.*

2.3 Text, number, date inputs

>> inputs.py

```
name  = st.text_input("Name", placeholder="Type here", max_chars=30)
pwd   = st.text_input("Password", type="password")
bio   = st.text_area("About you", height=120)
age   = st.number_input("Age", min_value=1, max_value=120, value=25, step=1)
marks = st.slider("Marks", 0, 100, 60)                  # single value
lo, hi = st.slider("Range", 0, 100, (20, 80))           # tuple -> range slider
grade = st.select_slider("Grade", ["D", "C", "B", "A", "A+"], value="B")
dob   = st.date_input("Date of birth", value=None)      # datetime.date (or None)
t     = st.time_input("Alarm time")                     # datetime.time
color = st.color_picker("Pick colour", "#ff4b4b")       # hex string
```

Widget	Returns
text_input / text_area	str ('' when empty)
number_input	int or float (type follows min/value/step)
slider	number, or tuple for a range
date_input / time_input	datetime.date / datetime.time (None allowed)
color_picker	'#RRGGBB' string

2.4 Choice widgets

>> choices.py

```
agree = st.checkbox("I agree")                          # bool
dark  = st.toggle("Dark mode")                          # bool, switch style
cls   = st.radio("Class", ["SL", "3A", "2A"], horizontal=True)
city  = st.selectbox("City", ["Pune", "Delhi", "Goa"], index=None,
                     placeholder="Choose a city")       # None until chosen
tags  = st.multiselect("Skills", ["Python", "SQL", "ML"], max_selections=2)
pick  = st.pills("Quick filter", ["All", "Open", "Closed"], default="All")
mode  = st.segmented_control("View", ["Table", "Chart"], default="Table")
```

Need	Use
Yes / No option	checkbox or toggle
Pick exactly one (2-5 options)	radio, pills or segmented_control
Pick one from a long list	selectbox
Pick many	multiselect (or pills with selection_mode='multi')
Pick from ordered values	select_slider

2.5 File, camera and audio inputs

>> uploads.py

```
f = st.file_uploader("Upload CSV", type=["csv"])          # UploadedFile or None
if f is not None:
    df = pd.read_csv(f)
    st.dataframe(df.head())
```

```python
files = st.file_uploader("Photos", type=["png", "jpg"], accept_multiple_files=True)
for im in files:
    st.image(im, caption=im.name, width=200)

pic = st.camera_input("Take a selfie")      # works on phone & laptop webcam
voice = st.audio_input("Record a message")  # microphone recording
```

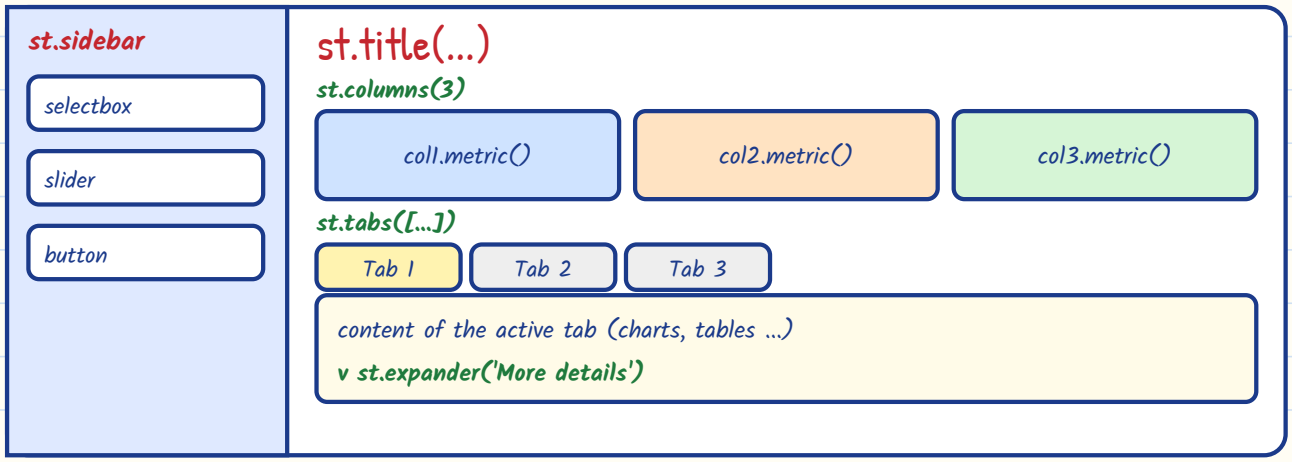
> **NOTE:** *Uploaded files are held in memory (default limit **200 MB** per file, change with `server.maxUploadSize`). The widget returns a file-like object: pass it straight to `pd.read_csv`, `PIL.Image.open`, etc.*

2.6 Editable tables - st.data_editor

>> data_editor.py

```python
df = pd.DataFrame({"Item": ["Pen", "Book"], "Qty": [10, 4], "Paid": [True, False]})
edited = st.data_editor(
    df, num_rows="dynamic",                    # user can add / delete rows
    column_config={"Qty": st.column_config.NumberColumn(min_value=0, step=1),
                   "Paid": st.column_config.CheckboxColumn("Paid?")},
    width="stretch", key="editor")
st.write("Total quantity:", edited["Qty"].sum())    # returns the EDITED DataFrame
```

2.7 Layout: sidebar, columns, tabs, containers



>> layout.py

```python
with st.sidebar:                                # left panel for controls
    city = st.selectbox("City", ["Pune", "Delhi"])

a, b, c = st.columns(3)                         # equal columns
a.metric("Sales", "Rs 4.2L", "+8%")
b.metric("Orders", 120)
with c:
    st.write("Any widget can go here")
left, right = st.columns([2, 1], gap="large")     # 2:1 width ratio

tab1, tab2 = st.tabs(["Table", "Chart"])
with tab1: st.dataframe(df)
with tab2: st.line_chart(df)

with st.expander("Show details", expanded=False):
    st.write("Hidden until clicked")
```

```
box = st.container(border=True, height=200)       # scrollable bordered box
box.write("Inside the container")

placeholder = st.empty()                          # one slot you can overwrite
placeholder.info("Loading...")
placeholder.success("Done")

with st.popover("Settings"):                      # small floating menu
    st.checkbox("Notifications")
```

>> dialog.py

```
@st.dialog("Confirm delete")                     # modal pop-up window
def confirm():
    st.write("Are you sure?")
    if st.button("Yes, delete"):
        st.session_state.deleted = True
        st.rerun()                                # closes dialog

if st.button("Delete record"):
    confirm()
```

2.8 Status and feedback messages

>> status.py

```
st.success("Saved!");  st.info("FYI");  st.warning("Careful");  st.error("Failed")
st.exception(ValueError("demo"))                  # shows traceback nicely
st.toast("Booking confirmed", icon=":material/check_circle:")

with st.spinner("Calculating..."):                # shows while block runs
    time.sleep(2)

bar = st.progress(0, text="Working")
for i in range(100):
    time.sleep(0.02); bar.progress(i + 1, text=f"{i+1}%")

with st.status("Downloading data...", expanded=True) as s:
    st.write("Fetching"); time.sleep(1)
    s.update(label="Done!", state="complete")

st.balloons()                                     # celebration  (st.snow() too)
```

2.9 Forms - batch the inputs

*Without a form, **every** keystroke or click reruns the script. Wrap inputs in `st.form` and nothing runs until the user presses the "submit" button - perfect for registration, search filters and long forms.*

>> form.py

```
with st.form("signup", clear_on_submit=True):
    name  = st.text_input("Name")
    email = st.text_input("Email")
    plan  = st.selectbox("Plan", ["Free", "Pro"])
    ok    = st.form_submit_button("Register")       # required inside a form

if ok:
    st.success(f"Welcome {name} ({plan})")
```

> **TIP:** Inside a form you can't use normal `st.button` and widgets don't trigger callbacks until submit. If you need a widget to change the form's contents (e.g. *number of passengers*), put that widget **outside** the form - exactly what we do in the railway project.

2.10 Callbacks, fragments and flow control

- `on_click` / `on_change` callbacks run **before** the script reruns - ideal for updating session_state.
- `st.rerun()` - stop now and immediately start a new run (after changing state).
- `st.stop()` - end the script here (e.g. when the user is not logged in).
- `@st.fragment` - only this function reruns when its own widgets change (no full-page rerun).
- `@st.fragment(run_every="5s")` - auto-refresh part of a page (live dashboards).

>> flow.py

```
import time, datetime, streamlit as st

if "user" not in st.session_state:
    st.warning("Please log in first")
    st.stop()                                   # nothing below this line runs

@st.fragment(run_every="5s")                    # refreshes itself every 5 seconds
def clock():
    st.metric("Server time", datetime.datetime.now().strftime("%H:%M:%S"))

clock()
```

2.11 Multipage apps

Method 1 (simple): create a `pages/` folder; each `.py` file becomes a sidebar page. Prefix with numbers to order them. **Method 2 (recommended):** `st.navigation` + `st.Page` give you full control (grouping, icons, login-dependent menus).

>> method 1 - folder

```
my_app/
|-- app.py
|-- pages/
      |-- 1_Dashboard.py
      |-- 2_Reports.py
```

>> app.py

```
# app.py  (method 2)
import streamlit as st

pages = {
    "Main": [st.Page("home.py", title="Home", icon=":material/home:", default=True),
             st.Page("search.py", title="Search", icon=":material/search:")],
    "Admin": [st.Page("admin.py", title="Admin", icon=":material/settings:")],
}
pg = st.navigation(pages)        # builds the sidebar menu
pg.run()                         # runs the chosen page

# inside any page:
st.page_link("search.py", label="Go to Search")
if st.button("Back"):
```

```
    st.switch_page("home.py")
```

> **REMEMBER:** `st.session_state` is shared by all pages of the same session, so it's the natural place to keep the logged-in user.

2.12 Config, theme and secrets

>> config.toml

```
# .streamlit/config.toml
[server]
port = 8501
maxUploadSize = 50              # MB

[theme]
primaryColor = "#c4262e"
backgroundColor = "#fffdf4"
secondaryBackgroundColor = "#e8eefc"
textColor = "#1b3a8a"
font = "sans serif"

[browser]
gatherUsageStats = false
```

>> secrets.toml

```
# .streamlit/secrets.toml   (add to .gitignore!)
OPENAI_KEY = "sk-xxxx"
[db]
user = "admin"
password = "secret"
```

>> using them

```
import streamlit as st
key = st.secrets["OPENAI_KEY"]
dbuser = st.secrets["db"]["user"]

st.set_page_config(page_title="My App", page_icon=":material/bolt:",
                   layout="wide",                    # use full browser width
                   initial_sidebar_state="expanded")   # MUST be the first st.* call
```

> **CAREFUL:** `st.set_page_config()` must be the **first** Streamlit command in the script, and only once per page. On Streamlit Community Cloud paste the secrets file contents in *App settings -> Secrets*.

2.13 Widget cheat sheet

Category	Calls
Buttons	button, download_button, link_button, form_submit_button, page_link
Text / numbers	text_input, text_area, number_input, slider, select_slider
Date / colour	date_input, time_input, color_picker
Choices	checkbox, toggle, radio, selectbox, multiselect, pills, segmented_control
Files / media	file_uploader, camera_input, audio_input
Tables	dataframe (selectable), data_editor, table

Category	Calls
Layout	sidebar, columns, tabs, expander, container, empty, popover, dialog
Status	success, info, warning, error, toast, spinner, progress, status
Chat	chat_message, chat_input, write_stream
Control	form, fragment, rerun, stop, navigation, Page, switch_page

3 Railway Reservation

end-to-end project: IRCTC-style booking system with Streamlit + SQLite

3.1 Problem statement

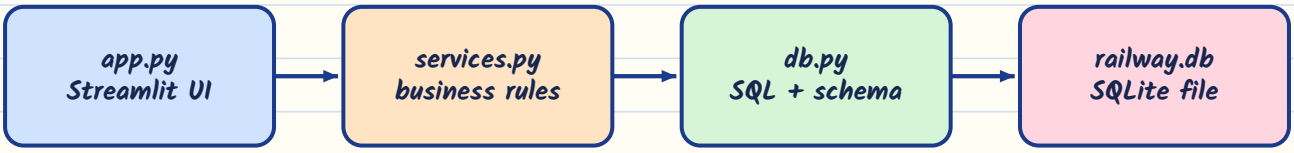
*Build a small **Indian Railway ticket reservation system**: passengers register, search trains between two stations, check seat availability, book tickets for several passengers, get a **10-digit PNR**, check PNR status, cancel with a refund, and an admin sees revenue and adds trains.*

Module	Features
Accounts	Register, login (salted PBKDF2 password hash), roles: user / admin
Search	From / To station codes, journey date (max 120 days ahead), class-wise availability
Booking	1-6 passengers, fare calculation, demo payment, CNF or WL status, downloadable ticket
PNR status	Public page: train details + every passenger's status
Cancellation	Time-based refund slabs; first waiting-list passengers get promoted to CNF
Admin	Revenue chart per train, add new train form

> **CAREFUL:** *This is a **learning project with sample data** (train numbers/timings are illustrative, fares and refund rules are simplified). It does not connect to IRCTC or take real payments.*

3.2 Architecture and folders

*Three layers keep the code clean: the **UI** only draws widgets, **services** hold all rules, **db** talks to SQLite. Because services have no `st.` calls they can be unit-tested without a browser.*



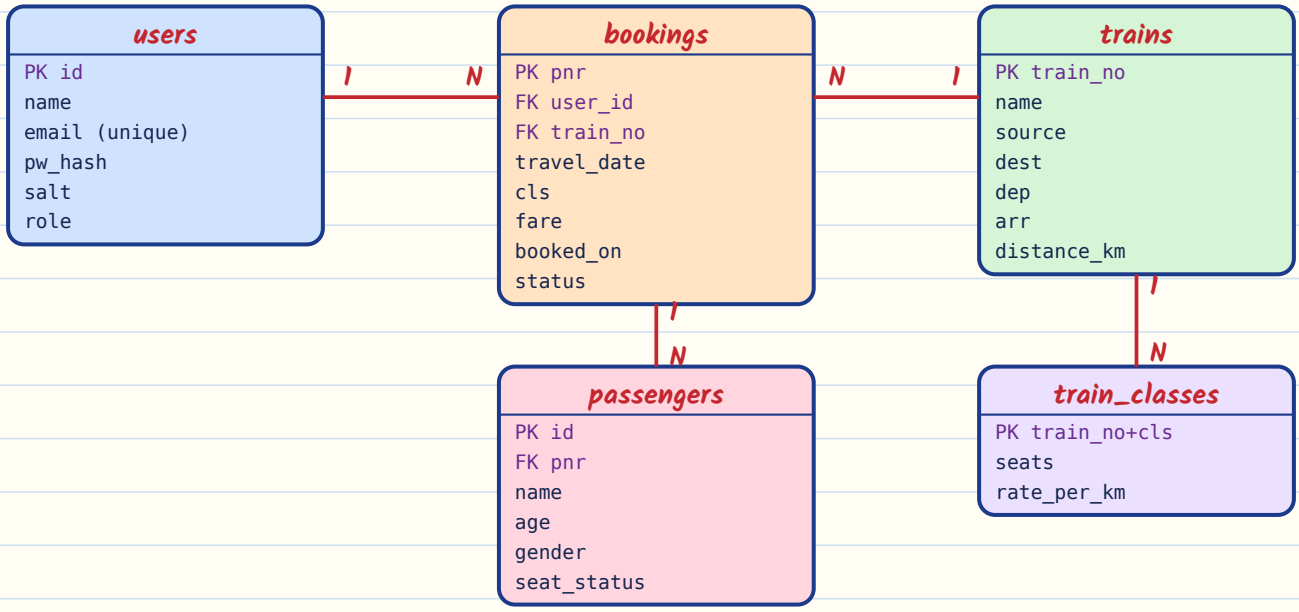
>> structure

```
railway_app/
|-- app.py          # pages: Home, PNR Status, Search & Book, My Bookings, Admin
|-- services.py     # register/login, availability, fare, book, cancel, revenue
|-- db.py           # connection, schema, sample trains, password hashing, PNR
|-- railway.db      # created automatically on first run
|-- requirements.txt   # streamlit, pandas
```

>> run

```
pip install streamlit pandas
streamlit run app.py
# demo admin login:  admin@rail.in / admin123
```

3.3 Database design



- **users** - one row per account; password stored as `pw_hash` + random `salt` (never plain text).
- **trains** and **train_classes** - a train offers several classes, each with seats and a per-km rate.
- **bookings** - one PNR = one booking; **passengers** - 1 to 6 rows per PNR with `seat_status` = CNF / WL / CAN.
- Foreign keys link tables (`PRAGMA foreign_keys = ON` turns enforcement on in SQLite).

3.4 Business rules

Rule	How it works in the code
Fare	(distance x rate_per_km + reservation fee) + 5% GST (AC classes only), x passengers
Availability	free = seats - CNF count. free > 0: AVAILABLE n. else WL if waiting < 3. else REGRET
Booking	first `free` passengers get CNF, the rest WL (e.g. 1 seat left, 2 passengers = CNF + WL)
PNR	random 10-digit number, retried until unique
Refund	>48h: fare - Rs 60/passenger. 12-48h: 75% of fare - flat. 4-12h: 50% - flat. <4h: nothing
Promotion	after a cancel, oldest WL passengers (by booking time) become CNF while seats are free

Worked example - 12951 Mumbai Rajdhani, class 3A, 2 passengers:

>> fare check (matches the code)

```
base per passenger = 1384 km x 1.45        = 2006.80
+ reservation fee                           =   40.00   ->  2046.80
+ 5% GST on 2046.80                         =  102.34   ->  2149.14
total for 2 passengers = 2149.14 x 2        =  4298.28
```

3.5 db.py – connection, schema, sample data

>> db.py (part 1) – schema

```
# db.py  -  SQLite layer for the railway system
import sqlite3, hashlib, os, random
from datetime import datetime

DB_PATH = "railway.db"

def get_conn():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA foreign_keys = ON")
    return conn

SCHEMA = """
CREATE TABLE IF NOT EXISTS users(
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL, email TEXT UNIQUE NOT NULL,
  pw_hash TEXT NOT NULL, salt TEXT NOT NULL,
  role TEXT NOT NULL DEFAULT 'user');
CREATE TABLE IF NOT EXISTS trains(
  train_no TEXT PRIMARY KEY, name TEXT NOT NULL,
  source TEXT NOT NULL, dest TEXT NOT NULL,
  dep TEXT NOT NULL, arr TEXT NOT NULL, distance_km INTEGER NOT NULL);
CREATE TABLE IF NOT EXISTS train_classes(
  train_no TEXT REFERENCES trains(train_no),
  cls TEXT NOT NULL, seats INTEGER NOT NULL, rate_per_km REAL NOT NULL,
  PRIMARY KEY(train_no, cls));
CREATE TABLE IF NOT EXISTS bookings(
  pnr TEXT PRIMARY KEY, user_id INTEGER REFERENCES users(id),
  train_no TEXT REFERENCES trains(train_no), travel_date TEXT NOT NULL,
  cls TEXT NOT NULL, fare REAL NOT NULL, booked_on TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'BOOKED');
CREATE TABLE IF NOT EXISTS passengers(
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  pnr TEXT REFERENCES bookings(pnr),
  name TEXT NOT NULL, age INTEGER NOT NULL, gender TEXT NOT NULL,
  seat_status TEXT NOT NULL);   -- 'CNF' | 'WL' | 'CAN'
"""
```

`sqlite3.Row` *lets us read columns by name (*`row['seats']`*).* `executescript` *runs many CREATE statements at once. Each table is created only* `IF NOT EXISTS`*, so the app can start safely many times.*

>> db.py (part 2) – seed data, hashing, PNR

```
SAMPLE_TRAINS = [  # sample data only - not the real timetable
 ("12951","Mumbai Rajdhani","MMCT","NDLS","17:00","08:32",1384),
 ("12301","Howrah Rajdhani","HWH","NDLS","16:55","10:00",1450),
 ("12627","Karnataka Express","SBC","NDLS","19:20","09:00",2370),
 ("12009","Shatabdi Express","MMCT","ADI","06:25","12:40",492),
 ("12123","Deccan Queen","CSMT","PUNE","17:10","20:25",192),
]
RATES = {"SL":0.55, "3A":1.45, "2A":2.10, "1A":3.60, "CC":1.60}

def hash_pw(password, salt=None):
    salt = salt or os.urandom(16).hex()
    h = hashlib.pbkdf2_hmac("sha256", password.encode(), bytes.fromhex(salt), 100_000)
    return h.hex(), salt

def init_db():
    with get_conn() as c:
        c.executescript(SCHEMA)
        if c.execute("SELECT COUNT(*) FROM trains").fetchone()[0] == 0:
```

```
            c.executemany("INSERT INTO trains VALUES (?,?,?,?,?,?,?)", SAMPLE_TRAINS)
            for t in SAMPLE_TRAINS:
                allowed = ["CC"] if t[0] in ("12009","12123") else ["SL","3A","2A","1A"]
                for cls in allowed:
                    c.execute("INSERT INTO train_classes VALUES (?,?,?,?)",
                              (t[0], cls, 6, RATES[cls]))   # 6 seats: easy to test WL
            h, s = hash_pw("admin123")
            c.execute("INSERT INTO users(name,email,pw_hash,salt,role) "
                      "VALUES ('Admin','admin@rail.in',?,?,'admin')", (h, s))

def new_pnr(conn):
    while True:
        pnr = str(random.randint(10**9, 10**10 - 1))
        if not conn.execute("SELECT 1 FROM bookings WHERE pnr=?", (pnr,)).fetchone():
            return pnr
```

*TIP: Passwords use **PBKDF2-HMAC-SHA256** with a random 16-byte salt and 100,000 iterations from Python's standard library. For production prefer `argon2-cffi` or `bcrypt` and more iterations, or use Streamlit's built-in `st.login()` (OpenID Connect).*

3.6 services.py - the business logic

>> services.py - register & login

```
# services.py  -  business rules (no Streamlit code here)
from datetime import datetime, timedelta
import pandas as pd
from db import get_conn, hash_pw, new_pnr

MAX_WL = 3          # max waiting-list tickets per class/date
GST_AC = 0.05       # 5% GST on AC classes (simplified)
RES_FEE = {"SL":20, "3A":40, "2A":50, "1A":60, "CC":40}

# ---------- users ----------
def register(name, email, password):
    h, s = hash_pw(password)
    try:
        with get_conn() as c:
            c.execute("INSERT INTO users(name,email,pw_hash,salt) VALUES(?,?,?,?)",
                      (name, email.lower().strip(), h, s))
        return True, "Registered! Please log in."
    except Exception:
        return False, "Email already registered."

def login(email, password):
    with get_conn() as c:
        u = c.execute("SELECT * FROM users WHERE email=?",
                      (email.lower().strip(),)).fetchone()
    if u and hash_pw(password, u["salt"])[0] == u["pw_hash"]:
        return dict(u)
    return None
```

`INSERT` *fails with* `IntegrityError` *when the email already exists (UNIQUE) - we catch it and return a friendly message.* `login` *re-hashes the typed password with the stored salt and compares.*

>> services.py - search & availability

```
# ---------- search ----------
def stations():
    with get_conn() as c:
        rows = c.execute("SELECT source s FROM trains UNION SELECT dest FROM trains")
        return sorted(r[0] for r in rows)
```

```
def count_trains():
    with get_conn() as c:
        return c.execute("SELECT COUNT(*) FROM trains").fetchone()[0]

def search_trains(src, dst):
    with get_conn() as c:
        return pd.read_sql("SELECT * FROM trains WHERE source=? AND dest=?",
                           c, params=(src, dst))

def class_info(train_no):
    with get_conn() as c:
        return pd.read_sql("SELECT cls, seats, rate_per_km FROM train_classes "
                           "WHERE train_no=?", c, params=(train_no,))

def availability(train_no, date, cls):
    """returns (status_text, confirmed_left, waiting_count)"""
    with get_conn() as c:
        seats = c.execute("SELECT seats FROM train_classes WHERE train_no=? AND cls=?",
                          (train_no, cls)).fetchone()["seats"]
        q = ("SELECT seat_status, COUNT(*) n FROM passengers p JOIN bookings b "
             "ON p.pnr=b.pnr WHERE b.train_no=? AND b.travel_date=? AND b.cls=? "
             "GROUP BY seat_status")
        cnt = {r["seat_status"]: r["n"] for r in c.execute(q, (train_no, date, cls))}
    left = seats - cnt.get("CNF", 0)
    wl = cnt.get("WL", 0)
    if left > 0:
        return f"AVAILABLE {left}", left, wl
    if wl < MAX_WL:
        return f"WL {wl + 1}", 0, wl
    return "REGRET / NOT AVAILABLE", 0, wl
```

`availability()` *counts CNF/WL passengers with one GROUP BY query, then returns a text status plus numbers the UI can reuse. Parameters use* `?` *placeholders - this blocks* ***SQL injection****.*

>> services.py - fare

```
# ---------- fare ----------
def calc_fare(distance, rate, cls, n_pass):
    base = distance * rate
    fee = RES_FEE[cls]
    gst = (base + fee) * GST_AC if cls != "SL" else 0
    return round((base + fee + gst) * n_pass, 2)
```

>> services.py - booking, PNR status, my bookings

```
# ---------- booking ----------
def book(user_id, train_no, date, cls, passengers):
    with get_conn() as c:
        t = c.execute("SELECT * FROM trains WHERE train_no=?", (train_no,)).fetchone()
        rate = c.execute("SELECT rate_per_km FROM train_classes WHERE train_no=? "
                         "AND cls=?", (train_no, cls)).fetchone()[0]
    status, left, wl = availability(train_no, date, cls)
    if status.startswith("REGRET"):
        return None, "No seats / waiting list available."
    extra = max(len(passengers) - left, 0)          # passengers who will be WL
    if wl + extra > MAX_WL:
        return None, "Not enough waiting-list space for all passengers."
    fare = calc_fare(t["distance_km"], rate, cls, len(passengers))
    with get_conn() as c:
        pnr = new_pnr(c)
        c.execute("INSERT INTO bookings VALUES (?,?,?,?,?,?,?,'BOOKED')",
                  (pnr, user_id, train_no, date, cls, fare,
                   datetime.now().strftime("%Y-%m-%d %H:%M")))
        for p in passengers:
            st_ = "CNF" if left > 0 else "WL"
            if left > 0: left -= 1
            c.execute("INSERT INTO passengers(pnr,name,age,gender,seat_status) "
```

```
                      "VALUES (?,?,?,?,?)", (pnr, p["name"], p["age"], p["gender"], st_))
    return pnr, f"Ticket booked. PNR {pnr}"

def pnr_status(pnr):
    with get_conn() as c:
        b = c.execute("SELECT b.*, t.name tname, t.source, t.dest, t.dep, t.arr "
                      "FROM bookings b JOIN trains t USING(train_no) WHERE pnr=?",
                      (pnr,)).fetchone()
        if not b:
            return None, None
        p = pd.read_sql("SELECT name, age, gender, seat_status FROM passengers "
                        "WHERE pnr=?", c, params=(pnr,))
    return dict(b), p

def my_bookings(user_id):
    with get_conn() as c:
        return pd.read_sql("SELECT pnr, train_no, travel_date, cls, fare, status "
                           "FROM bookings WHERE user_id=? ORDER BY booked_on DESC",
                           c, params=(user_id,))
```

> **REMEMBER:** *Everything for one booking happens in one connection block: if any INSERT fails the `with get_conn()` context rolls the whole transaction back, so you never get a PNR without passengers.*

>> services.py – cancellation, refund, waiting-list promotion

```
# ---------- cancellation ----------
def refund_amount(fare, n_pass, dep_dt, now=None):
    now = now or datetime.now()
    hrs = (dep_dt - now).total_seconds() / 3600
    if hrs < 4:
        return 0.0
    cut = 0.5 if hrs < 12 else 0.25 if hrs < 48 else 0.0
    flat = 60 * n_pass
    return round(max(fare * (1 - cut) - flat, 0), 2)

def cancel(pnr, user_id):
    b, p = pnr_status(pnr)
    if not b or b["user_id"] != user_id:
        return False, "PNR not found for this account."
    if b["status"] == "CANCELLED":
        return False, "Already cancelled."
    dep_dt = datetime.strptime(f"{b['travel_date']} {b['dep']}", "%Y-%m-%d %H:%M")
    refund = refund_amount(b["fare"], len(p), dep_dt)
    with get_conn() as c:
        c.execute("UPDATE bookings SET status='CANCELLED' WHERE pnr=?", (pnr,))
        c.execute("UPDATE passengers SET seat_status='CAN' WHERE pnr=?", (pnr,))
    promote_waitlist(b["train_no"], b["travel_date"], b["cls"])
    return True, f"Cancelled. Refund: Rs {refund}"

def promote_waitlist(train_no, date, cls):
    """oldest WL passengers become CNF while seats are free"""
    _, left, _ = availability(train_no, date, cls)
    with get_conn() as c:
        wl = c.execute("SELECT p.id FROM passengers p JOIN bookings b ON p.pnr=b.pnr "
                       "WHERE b.train_no=? AND b.travel_date=? AND b.cls=? "
                       "AND p.seat_status='WL' ORDER BY b.booked_on, p.id",
                       (train_no, date, cls)).fetchall()
        for row in wl[:max(left, 0)]:
            c.execute("UPDATE passengers SET seat_status='CNF' WHERE id=?", (row[0],))
```

>> services.py – admin helpers

```
# ---------- admin ----------
def revenue_by_train():
    with get_conn() as c:
```

```
        return pd.read_sql("SELECT train_no, COUNT(*) bookings, SUM(fare) revenue "
                           "FROM bookings WHERE status='BOOKED' GROUP BY train_no", c)

def add_train(no, name, src, dst, dep, arr, dist, classes):
    with get_conn() as c:
        c.execute("INSERT INTO trains VALUES (?,?,?,?,?,?,?)",
                  (no, name, src, dst, dep, arr, dist))
        for cls, seats, rate in classes:
            c.execute("INSERT INTO train_classes VALUES (?,?,?,?)",
                      (no, cls, seats, rate))
```

3.7 app.py - the Streamlit interface

>> app.py - setup, session defaults, ticket text

```
# app.py  -  Streamlit UI for the Railway Reservation System
import streamlit as st
from datetime import date, timedelta
import db, services as svc

st.set_page_config(page_title="Indian Rail Reservation", page_icon=":material/train:", layout="wide")
db.init_db()                                  # creates tables + sample data once

# ---------- session defaults ----------
st.session_state.setdefault("user", None)
st.session_state.setdefault("pending", None)   # booking waiting for payment

def ticket_text(b, p):
    lines = [f"PNR: {b['pnr']}", f"Train: {b['train_no']} {b['tname']}",
             f"{b['source']} -> {b['dest']}  on {b['travel_date']}  Class {b['cls']}",
             f"Fare: Rs {b['fare']}  Status: {b['status']}", "Passengers:"]
    lines += [f" - {r['name']} ({r['age']}/{r['gender']}) {r['seat_status']}"
              for _, r in p.iterrows()]
    return "\n".join(lines)
```

`db.init_db()` *runs on every rerun but is cheap and idempotent.* `setdefault` *creates* `user` *and* `pending` *once per session.*

>> app.py - sidebar: login / register / navigation

```
# ---------- sidebar: auth + navigation ----------
with st.sidebar:
    st.title(":material/train: IRCTC Lite")
    user = st.session_state.user
    if user:
        st.success(f"Hello, {user['name']}")
        if st.button("Logout", width="stretch"):
            st.session_state.user = None
            st.session_state.pending = None
            st.rerun()
    else:
        tab_in, tab_up = st.tabs(["Login", "Register"])
        with tab_in:
            with st.form("login"):
                em = st.text_input("Email")
                pw = st.text_input("Password", type="password")
                if st.form_submit_button("Login"):
                    u = svc.login(em, pw)
                    if u:
                        st.session_state.user = u
                        st.rerun()
                    st.error("Invalid credentials")
        with tab_up:
            with st.form("register"):
                nm = st.text_input("Full name")
                em2 = st.text_input("Email ")
```

```
                pw2 = st.text_input("Password ", type="password")
                if st.form_submit_button("Create account"):
                    if nm and em2 and len(pw2) >= 6:
                        ok, msg = svc.register(nm, em2, pw2)
                        (st.success if ok else st.error)(msg)
                    else:
                        st.warning("Fill all fields (password >= 6 chars).")

    pages = ["Home", "PNR Status"]
    if user:
        pages += ["Search & Book", "My Bookings"]
        if user["role"] == "admin":
            pages += ["Admin"]
    page = st.radio("Go to", pages)
```

*Login form -> `svc.login()` -> store the user dict in `st.session_state.user` -> `st.rerun()` so the sidebar immediately shows "Hello, name" and the extra pages. The **page list depends on login and role**, giving simple access control.*

>> app.py - Home and PNR Status pages

```
# ---------- pages ----------
if page == "Home":
    st.title("Indian Railway Reservation System")
    st.write("Search trains, book tickets, check PNR, cancel and get refunds.")
    c1, c2, c3 = st.columns(3)
    c1.metric("Trains", svc.count_trains())
    c2.metric("Classes", "SL / 3A / 2A / 1A / CC")
    c3.metric("Waiting list", f"max {svc.MAX_WL} per class")
    st.info("Demo admin login: admin@rail.in / admin123 - sample data only.")

elif page == "PNR Status":
    st.header("PNR Status")
    pnr = st.text_input("Enter 10-digit PNR", max_chars=10)
    if st.button("Check status") and pnr:
        b, p = svc.pnr_status(pnr)
        if b is None:
            st.error("PNR not found")
        else:
            st.subheader(f"{b['train_no']} - {b['tname']}")
            st.write(f"{b['source']} -> {b['dest']} | {b['travel_date']} "
                     f"{b['dep']} | Class {b['cls']} | Status: **{b['status']}**")
            st.dataframe(p, width="stretch", hide_index=True)
```

>> app.py - Search & Book page (the heart of the app)

```
elif page == "Search & Book":
    st.header("Search & Book")
    stn = svc.stations()
    c1, c2, c3 = st.columns(3)
    src = c1.selectbox("From", stn, index=stn.index("MMCT"))
    dst = c2.selectbox("To", stn, index=stn.index("NDLS"))
    dt = c3.date_input("Journey date", min_value=date.today(),
                       max_value=date.today() + timedelta(days=120),
                       value=date.today() + timedelta(days=7))
    trains = svc.search_trains(src, dst)
    if trains.empty:
        st.warning("No direct trains for this pair (try MMCT -> NDLS).")
        st.stop()
    st.dataframe(trains[["train_no", "name", "dep", "arr", "distance_km"]],
                 width="stretch", hide_index=True)

    tno = st.selectbox("Select train", trains["train_no"],
                       format_func=lambda x: f"{x} - "
                       f"{trains.set_index('train_no').loc[x, 'name']}")
    info = svc.class_info(tno)
```

```
    st.subheader("Availability")
    cols = st.columns(len(info))
    for col, (_, r) in zip(cols, info.iterrows()):
        status, _, _ = svc.availability(tno, dt.isoformat(), r["cls"])
        col.metric(r["cls"], status)

    cls = st.radio("Class", list(info["cls"]), horizontal=True)
    n = st.number_input("No. of passengers", 1, 6, 1)   # outside form -> dynamic rows
    dist = int(trains.set_index("train_no").loc[tno, "distance_km"])
    rate = float(info.set_index("cls").loc[cls, "rate_per_km"])
    st.caption(f"Estimated fare: Rs {svc.calc_fare(dist, rate, cls, n)}")

    with st.form("passengers"):
        plist = []
        for i in range(n):
            a, b, c = st.columns([3, 1, 1])
            plist.append({"name": a.text_input(f"Passenger {i+1} name", key=f"n{i}"),
                          "age": b.number_input("Age", 1, 120, 25, key=f"a{i}"),
                          "gender": c.selectbox("Gender", ["M", "F", "O"], key=f"g{i}")})
        go = st.form_submit_button("Review booking")
    if go:
        if all(p["name"].strip() for p in plist):
            st.session_state.pending = dict(tno=tno, date=dt.isoformat(), cls=cls,
                                            plist=plist,
                                            fare=svc.calc_fare(dist, rate, cls, n))
        else:
            st.error("Enter every passenger's name.")

    pend = st.session_state.pending
    if pend:
        st.divider()
        st.write(f"**Pay Rs {pend['fare']}** for {len(pend['plist'])} passenger(s) "
                 f"on {pend['tno']} ({pend['date']}, {pend['cls']})")
        if st.button("Confirm & Pay (demo)", type="primary"):
            with st.spinner("Processing payment..."):
                pnr, msg = svc.book(st.session_state.user["id"], pend["tno"],
                                    pend["date"], pend["cls"], pend["plist"])
            st.session_state.pending = None
            if pnr:
                st.success(msg)
                st.balloons()
                b, p = svc.pnr_status(pnr)
                st.dataframe(p, width="stretch", hide_index=True)
                st.download_button("Download ticket", ticket_text(b, p),
                                   file_name=f"ticket_{pnr}.txt")
            else:
                st.error(msg)
```

- *Number of passengers sits **outside** the form so the form can draw that many rows.*
- *Each passenger widget has a unique `key` (`n0`, `a0`, `g0`, ...) to avoid duplicate-ID errors.*
- *Form submit only stores a `pending` dict; a second **Confirm & Pay** button performs the booking - a two-step flow using session_state.*
- *`st.balloons()`, `st.download_button()` give instant feedback and a text ticket.*

>> app.py - My Bookings and Admin pages

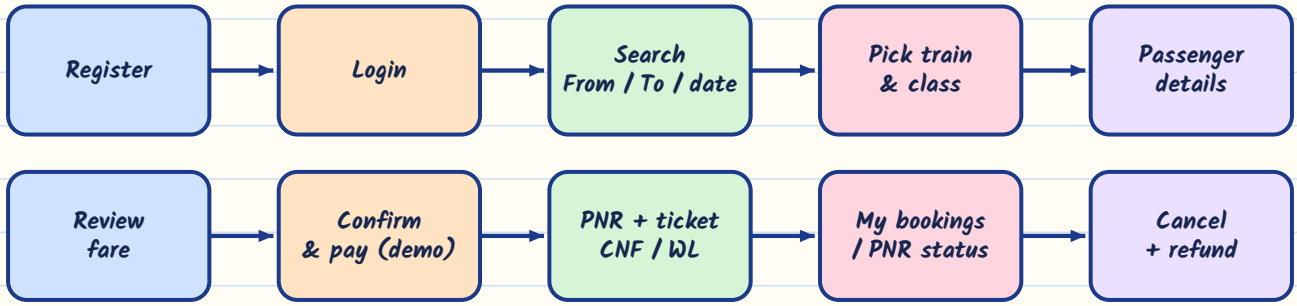
```
elif page == "My Bookings":
    st.header("My Bookings")
    if "flash" in st.session_state:                 # message saved before st.rerun()
        st.success(st.session_state.pop("flash"))
    df = svc.my_bookings(st.session_state.user["id"])
    if df.empty:
        st.info("No bookings yet.")
```

```
    else:
        st.dataframe(df, width="stretch", hide_index=True)
        active = df[df["status"] == "BOOKED"]["pnr"]
        if len(active):
            pnr = st.selectbox("Cancel which PNR?", active)
            if st.button("Cancel ticket"):
                ok, msg = svc.cancel(pnr, st.session_state.user["id"])
                if ok:
                    st.session_state.flash = msg
                    st.rerun()                      # refresh the table
                st.error(msg)

elif page == "Admin":
    st.header("Admin dashboard")
    rev = svc.revenue_by_train()
    if rev.empty:
        st.info("No revenue yet.")
    else:
        st.bar_chart(rev.set_index("train_no")["revenue"])
        st.dataframe(rev, width="stretch", hide_index=True)
    with st.expander("Add a new train"):
        with st.form("addtrain"):
            c1, c2 = st.columns(2)
            no = c1.text_input("Train no"); nm = c2.text_input("Name")
            s_ = c1.text_input("Source code"); d_ = c2.text_input("Dest code")
            dep = c1.text_input("Departure HH:MM", "10:00")
            arr = c2.text_input("Arrival HH:MM", "18:00")
            dist = st.number_input("Distance km", 1, 4000, 500)
            if st.form_submit_button("Add"):
                try:
                    svc.add_train(no, nm, s_.upper(), d_.upper(), dep, arr, dist,
                                  [("SL", 72, 0.55), ("3A", 64, 1.45)])
                    st.success("Train added")
                except Exception as e:
                    st.error(f"Could not add: {e}")
```

TIP: Why the `flash` message? `st.rerun()` clears everything drawn so far. We save the success text in `session_state`, rerun, then show and `pop` it on the next run.

3.8 User journey



3.9 Testing the app

*Seed data gives every class only **6 seats**, so you can see waiting lists quickly: book 5 seats, then 2 more -> second booking shows `CNF` + `WL`. Cancel the first booking and refresh "PNR status": the WL passenger becomes `CNF`.*

Streamlit ships a headless test tool, **AppTest**, that clicks widgets without a browser:

>> test_app.py (run: python test_app.py)

```
from streamlit.testing.v1 import AppTest

at = AppTest.from_file("app.py", default_timeout=30).run()
assert not at.exception                          # app starts without errors

at.sidebar.radio[0].set_value("PNR Status").run()   # open a page
at.text_input[0].set_value("1234567890")
at.button[0].click().run()                          # press 'Check status'
assert at.error[0].value == "PNR not found"
print("test passed")
```

I ran the complete register -> login -> book -> cancel -> PNR flow this way while preparing these notes.

3.10 Ideas to extend the project

- **Routes & stops:** a `train_route(train_no, station, seq, km, arr, dep)` table so passengers can board/alight at intermediate stations.
- **RAC, Tatkal, quotas** (ladies, senior citizen) and real **berth allocation** (Lower / Middle / Upper / Side).
- **Date-wise inventory table** and a daily job that creates future availability.
- Real payment gateway (e.g. Razorpay test mode), e-mail / SMS ticket, PDF ticket with `reportlab` + QR code.
- Move SQLite -> PostgreSQL; use `st.connection("sql")`; add `st.login()` OIDC authentication.
- Charts for admin: bookings per day (`st.line_chart`), class mix (Plotly pie), occupancy heat-map.
- Deploy with Docker (chapter 4) and add GitHub Actions running AppTest on every push.

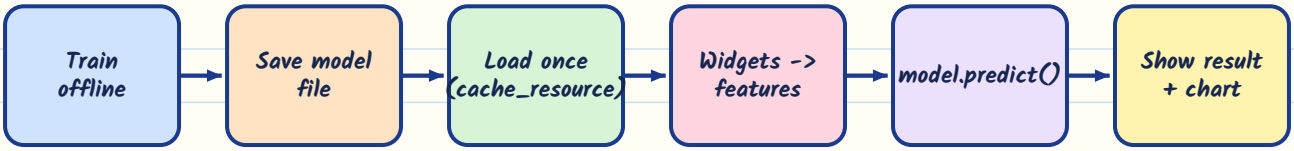
> **CAREFUL:** Security checklist for any real app: HTTPS, parameterised SQL only, strong password hashing, rate-limit login attempts, never keep secrets in code, validate every input, log errors without leaking data.

4 ML & DL Deployment

scikit-learn, TensorFlow/PyTorch models, caching, Docker, cloud

4.1 The deployment pattern

*A Streamlit ML app is **not** where you train - it is where users "use" the model. The recipe is always the same:*



- *Train in a notebook / script, **never inside** the Streamlit script (it reruns constantly).*
- *Save the **whole pipeline** (scaler + encoder + model) so preprocessing is identical in production.*
- *Load with `@st.cache_resource` so the file is read once and shared by all users.*
- *Build inputs with widgets, put them in a DataFrame with the **same columns/order** as training, then predict.*
- *Deploy: GitHub + Community Cloud, Docker anywhere, or Hugging Face Spaces.*

4.2 Saving and loading models

Library	Save	Load
scikit-learn	`joblib.dump(model, 'm.joblib')`	`joblib.load('m.joblib')`
Keras / TF	`model.save('m.keras')`	`tf.keras.models.load_model('m.keras')`
PyTorch	`torch.save(model.state_dict(), 'm.pt')`	`model.load_state_dict(torch.load('m.pt', map_location='cpu')); model.eval()`
XGBoost	`model.save_model('m.json')`	`xgb.XGBClassifier().load_model('m.json')`
ONNX	`export from any framework`	`onnxruntime.InferenceSession('m.onnx')` (light & fast)
HF Transformers	`pretrained from the Hub`	`pipeline('sentiment-analysis')` (cache it!)

CAREFUL: *Never `pickle.load` / `joblib.load` a file from an **untrusted source** - it can execute arbitrary code. Also load with the **same library versions** used for training (pin them in requirements.txt).*

4.3 Example 1 - scikit-learn classifier

Step 1: train once and save a "bundle" (model + feature names + class names + slider ranges + accuracy).

>> train_model.py

```
# train_model.py  -  run ONCE offline:  python train_model.py
import joblib
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

iris = load_iris(as_frame=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42, stratify=iris.target)

model = RandomForestClassifier(n_estimators=200, random_state=42).fit(X_tr, y_tr)
acc = accuracy_score(y_te, model.predict(X_te))
print("Test accuracy:", round(acc, 3))

joblib.dump({"model": model,
             "features": list(iris.data.columns),
             "classes": list(iris.target_names),
             "ranges": {c: (float(iris.data[c].min()), float(iris.data[c].max()))
                        for c in iris.data.columns},
             "accuracy": float(acc)}, "iris_model.joblib")
```

Step 2: the Streamlit app. Sliders get their min/max from the training data, and there is a batch mode for CSV files.

>> ml_app.py

```
# ml_app.py  -  streamlit run ml_app.py
import joblib
import pandas as pd
import streamlit as st

st.set_page_config(page_title="Iris Classifier", layout="centered")

@st.cache_resource                        # load the model ONCE per server process
def load_bundle():
    return joblib.load("iris_model.joblib")

bundle = load_bundle()
model, feats, classes = bundle["model"], bundle["features"], bundle["classes"]

st.title("Iris Flower Classifier")
st.caption(f"RandomForest | test accuracy {bundle['accuracy']:.1%}")

tab1, tab2, tab3 = st.tabs(["Single prediction", "Batch (CSV)", "Model info"])

with tab1:
    cols = st.columns(2)
    values = {}
    for i, f in enumerate(feats):
        lo, hi = bundle["ranges"][f]
        values[f] = cols[i % 2].slider(f, lo, hi, (lo + hi) / 2, step=0.1)
    if st.button("Predict", type="primary"):
        X = pd.DataFrame([values])[feats]            # same column order as training
        proba = model.predict_proba(X)[0]
        st.success(f"Predicted species: **{classes[proba.argmax()]}**")
        st.bar_chart(pd.Series(proba, index=classes, name="probability"))
```

```
with tab2:
    up = st.file_uploader("Upload CSV with columns: " + ", ".join(feats), type="csv")
    if up:
        df = pd.read_csv(up)
        missing = [c for c in feats if c not in df.columns]
        if missing:
            st.error(f"Missing columns: {missing}")
        else:
            df["prediction"] = [classes[i] for i in model.predict(df[feats])]
            st.dataframe(df, width="stretch")
            st.download_button("Download results", df.to_csv(index=False),
                               "predictions.csv", "text/csv")

with tab3:
    imp = pd.Series(model.feature_importances_, index=feats).sort_values()
    st.subheader("Feature importance")
    st.bar_chart(imp, horizontal=True)
```

*I trained this on the 150-row Iris dataset and ran the app headlessly with AppTest - clicking **Predict** returned a species with probabilities (test accuracy about 0.90 on the held-out 20%).*

- `@st.cache_resource` -> `joblib.load` *happens once, not on every slider move.*
- `pd.DataFrame([values])[feats]` *guarantees the column order the model expects.*
- `predict_proba` + `st.bar_chart` *shows **confidence**, not just a label.*
- *Batch tab: check required columns first, then predict all rows and offer* `download_button`.
- *Run:* `python train_model.py` *then* `streamlit run ml_app.py`.

4.4 Good practices for ML apps

>> pipeline.py

```
# 1) keep preprocessing INSIDE the saved model (no training-serving skew)
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=500)).fit(X_tr, y_tr)
joblib.dump(pipe, "pipe.joblib")          # app only needs: pipe.predict(df)
```

>> retrain + evaluation

```
# 2) optional: let users retrain small models inside the app
n_trees = st.slider("Trees", 10, 300, 100)
if st.button("Train model"):
    with st.spinner("Training..."):
        st.session_state.model = RandomForestClassifier(n_estimators=n_trees).fit(X_tr, y_tr)
    st.success("Done")

# 3) show evaluation visuals
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay
fig, ax = plt.subplots()
ConfusionMatrixDisplay.from_estimator(model, X_te, y_te, ax=ax)
st.pyplot(fig)
```

- ***Validate inputs**: ranges, missing values, wrong file format - show* `st.error` *instead of a crash.*
- *Cache **data** with* `cache_data`, ***models** with* `cache_resource`*; never load inside a button branch every time.*
- *Use* `st.form` *when a prediction needs many inputs, so it runs once on submit.*

- *Explain predictions: feature importance, SHAP plots (`st.pyplot`), nearest examples.*
- *Log predictions (time, inputs, output) to a file/DB to monitor drift later.*
- *Show a short **model card**: data, accuracy, limits, and a disclaimer for medical / financial use.*

4.5 Example 2 - deep learning image classifier

*MobileNetV2 trained on ImageNet (1000 classes) is small enough for free hosting. The app accepts an upload **or** a camera photo.*

>> dl_app.py (TensorFlow / Keras)

```
# dl_app.py  -  image classifier (MobileNetV2, ImageNet)   streamlit run dl_app.py
import numpy as np
import streamlit as st
import tensorflow as tf
from PIL import Image

st.set_page_config(page_title="Image Classifier")

@st.cache_resource(show_spinner="Loading model...")
def load_model():
    return tf.keras.applications.MobileNetV2(weights="imagenet")   # ~14 MB download

def predict(img: Image.Image, model, top=5):
    x = np.array(img.convert("RGB").resize((224, 224)), dtype="float32")
    x = tf.keras.applications.mobilenet_v2.preprocess_input(x[None, ...])
    preds = model.predict(x, verbose=0)
    return tf.keras.applications.mobilenet_v2.decode_predictions(preds, top=top)[0]

st.title("What is in this picture?")
model = load_model()

src = st.radio("Image source", ["Upload", "Camera"], horizontal=True)
file = (st.file_uploader("Choose an image", type=["jpg", "jpeg", "png"])
        if src == "Upload" else st.camera_input("Take a photo"))

if file:
    img = Image.open(file)
    st.image(img, width=320)
    with st.spinner("Predicting..."):
        results = predict(img, model)
    st.subheader(f"Prediction: {results[0][1].replace('_', ' ')}")
    for _, label, score in results:
        st.write(f"{label.replace('_', ' ')}  -  {score:.1%}")
        st.progress(float(score))
```

- `preprocess_input` *scales pixels the way MobileNetV2 was trained (-1 to 1) - skipping it ruins accuracy.*
- `x[None, ...]` *adds the **batch dimension**: shape (1, 224, 224, 3).*
- `decode_predictions` *converts class numbers into readable names with scores.*
- `st.progress(score)` *draws a quick confidence bar; `width=320` keeps the image small.*

>> same idea with PyTorch

```
# PyTorch version of the model loader (same Streamlit UI works)
import torch, torchvision
from PIL import Image
import streamlit as st
```

```
@st.cache_resource
def load_torch():
    w = torchvision.models.MobileNet_V2_Weights.DEFAULT
    m = torchvision.models.mobilenet_v2(weights=w).eval()
    return m, w.transforms(), w.meta["categories"]

def predict_torch(img: Image.Image):
    model, tfm, cats = load_torch()
    x = tfm(img.convert("RGB")).unsqueeze(0)
    with torch.no_grad():
        p = torch.softmax(model(x)[0], dim=0)
    top = torch.topk(p, 5)
    return [(cats[i], float(s)) for s, i in zip(top.values, top.indices)]
```

*NOTE: The first run **downloads weights** from the internet and needs the library installed (`pip install tensorflow-cpu pillow` or `torch torchvision`). Check which Python versions TensorFlow/PyTorch support - they sometimes lag behind the newest Python. I verified the app structure and syntax; run it locally to see predictions.*

4.6 Example 3 - train your own CNN (MNIST digits)

>> train_cnn.py

```
# train_cnn.py
import tensorflow as tf
(x_tr, y_tr), (x_te, y_te) = tf.keras.datasets.mnist.load_data()
x_tr, x_te = x_tr[..., None] / 255.0, x_te[..., None] / 255.0

model = tf.keras.Sequential([
    tf.keras.layers.Input((28, 28, 1)),
    tf.keras.layers.Conv2D(32, 3, activation="relu"),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")])
model.compile("adam", "sparse_categorical_crossentropy", metrics=["accuracy"])
model.fit(x_tr, y_tr, epochs=3, validation_split=0.1)
print(model.evaluate(x_te, y_te))
model.save("mnist_cnn.keras")
```

>> digit_app.py

```
# digit_app.py
import numpy as np, streamlit as st, tensorflow as tf
from PIL import Image

@st.cache_resource
def load():
    return tf.keras.models.load_model("mnist_cnn.keras")

st.title("Handwritten digit recognizer")
f = st.file_uploader("Upload a digit photo/PNG", type=["png", "jpg", "jpeg"])
if f:
    img = Image.open(f).convert("L").resize((28, 28))
    arr = np.array(img, dtype="float32") / 255.0
    if arr.mean() > 0.5:                # dark ink on white paper -> invert
        arr = 1 - arr
    probs = load().predict(arr[None, ..., None], verbose=0)[0]
    c1, c2 = st.columns(2)
    c1.image(img.resize((140, 140)), caption="what the model sees")
    c2.metric("Predicted digit", int(probs.argmax()), f"{probs.max():.1%} sure")
```

```
    st.bar_chart(probs)
```

Tip: LLM / transformer models work the same way - `@st.cache_resource def get_pipe(): return pipeline('sentiment-analysis')` from Hugging Face `transformers`, then call it with the text from `st.text_area`.

4.7 Performance and memory

- *Cache models with `cache_resource`; cache heavy preprocessing with `cache_data`.*
- *Prefer small models: MobileNet, DistilBERT, quantised / ONNX / TFLite versions.*
- *Free hosting has limited RAM and **no GPU** - choose CPU-friendly wheels (`tensorflow-cpu`, torch CPU).*
- *Import heavy libraries inside the cached loader if they are used on only one page.*
- *Resize big uploaded images before inference; set `server.maxUploadSize` lower.*
- *Large model file (>100 MB)? GitHub blocks it - use Git LFS or download at start-up (Hugging Face Hub / cloud bucket) inside the cached loader.*

4.8 Deploying: where and how

Option	Good for	How
Streamlit Community Cloud	Free public demos, student projects	Push app to GitHub -> share.streamlit.io -> pick repo/branch/file -> Deploy. Secrets in app settings. Redeploys on every push.
Hugging Face Spaces	ML community demos	Create a Space (Docker SDK works for any Streamlit app), push files.
Docker on a VM	Full control, private apps	Build image, run on AWS EC2 / Azure VM / any server behind Nginx.
Managed containers	Auto-scaling	Google Cloud Run, AWS App Runner/ECS, Azure Container Apps, Render, Railway, Fly.io.

>> requirements.txt

```
streamlit
scikit-learn
pandas
joblib
# freeze real versions:  pip freeze > requirements.txt   (or pin: scikit-learn==1.8.0)
```

>> Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8501
HEALTHCHECK CMD curl --fail http://localhost:8501/_stcore/health
ENTRYPOINT ["streamlit", "run", "ml_app.py", \
            "--server.port=8501", "--server.address=0.0.0.0"]
```

>> build & run

```
docker build -t iris-app .
docker run -p 8501:8501 iris-app          # open http://localhost:8501
```

- Streamlit uses **WebSockets** - your proxy / load balancer must allow them (Nginx: `Upgrade` headers).
- Bind to `0.0.0.0` inside containers, keep secrets in environment variables or `st.secrets`.
- Add `.gitignore` for `venv/`, `.streamlit/secrets.toml`, `*.db`; add `.dockerignore` too.

4.9 Common errors

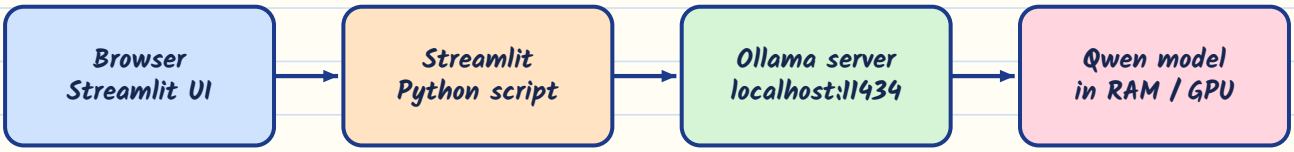
Problem	Cause -> Fix
ModuleNotFoundError after deploy	Package missing from requirements.txt -> add it and redeploy
FileNotFoundError: model	Wrong relative path -> run from project root or use Path(__file__).parent
Feature names mismatch (sklearn)	DataFrame columns/order differ from training -> use the saved feature list
Unpickle / version warnings	Different scikit-learn version -> pin the training version
App very slow on every click	Model loaded inside script body -> use @st.cache_resource
Out of memory / app restarts	Model or data too big -> smaller model, lazy load, more RAM
DuplicateWidgetID	Same widget twice -> add unique key=

5 Ollama + Qwen Chatbot

run an LLM locally and chat with it through Streamlit

5.1 What are Ollama and Qwen?

- ***Ollama*** - a free tool that downloads and runs open-weight LLMs ***on your own computer*** and exposes them through a local API at `http://localhost:11434`.
- ***Qwen*** (pronounced roughly *"chwen"*) - open-weight language-model family from Alibaba Cloud; strong at reasoning, coding and multilingual text. Several generations exist (qwen3, qwen3.5 ...).
- Why local? ***Privacy*** (data never leaves your PC), ***no API bill***, works ***offline***, and you can experiment freely.



> ***TIP:*** You wrote *owen* - the model is spelled ***Qwen***. Model names change quickly, so always confirm the latest tags on ***ollama.com/library*** (search 'qwen') before pulling.

5.2 Install and download a model

>> terminal

```
# Windows / macOS: download the installer from  https://ollama.com/download
# Linux:
curl -fsSL https://ollama.com/install.sh | sh

ollama --version
ollama pull qwen3.5:9b        # download (a few GB, one time)
ollama run qwen3.5:9b         # chat in the terminal  (type /bye to quit)
ollama list                   # models on disk
ollama ps                     # models loaded in memory right now
ollama show qwen3.5:9b        # details: size, context length, capabilities
ollama rm qwen3.5:9b          # delete a model
ollama serve                  # start the server manually (the desktop app does it for you)
```

Model size	Rough memory	Notes
~1-2B (qwen3:1.7b)	2-4 GB	very light, simple tasks, fast on CPU
~4B (qwen3:4b, qwen3.5:4b)	4-8 GB	good laptop choice
~8-9B (qwen3:8b, qwen3.5:9b)	8-16 GB	best balance of quality and speed
14B-35B	16-32 GB+	needs a strong PC or GPU

> **CAREFUL:** *Memory numbers are rule-of-thumb for the default 4-bit builds. Newer Qwen releases need a **newer Ollama version** - if `pull` complains, update Ollama. If a tag in these notes does not exist any more, run `ollama list` / browse the library and change the model name in the code.*

>> test the REST API (works in any language)

```
curl http://localhost:11434/api/chat -d '{
  "model": "qwen3.5:9b",
  "messages": [{"role": "user", "content": "Explain Streamlit in one line"}],
  "stream": false
}'
```

5.3 The Ollama Python client

>> install

```
pip install ollama streamlit
```

>> ollama_basics.py

```
import ollama

# 1) simple (blocking) call
r = ollama.chat(model="qwen3.5:9b",
                messages=[{"role": "user", "content": "Capital of Maharashtra?"}])
print(r.message.content)

# 2) streaming: tokens arrive one by one
for chunk in ollama.chat(model="qwen3.5:9b", stream=True,
                         messages=[{"role": "user", "content": "Write a haiku on rain"}]):
    print(chunk.message.content, end="", flush=True)

# 3) thinking models: reasoning arrives in a separate field
for chunk in ollama.chat(model="qwen3.5:9b", think=True, stream=True,
                         messages=[{"role": "user", "content": "17 x 23 = ?"}]):
    if chunk.message.thinking: print(chunk.message.thinking, end="")   # reasoning trace
    elif chunk.message.content: print(chunk.message.content, end="")   # final answer

# 4) options and utilities
ollama.chat(model="qwen3.5:9b", messages=[...], options={"temperature": 0.2, "num_ctx": 8192})
print([m.model for m in ollama.list().models])
```

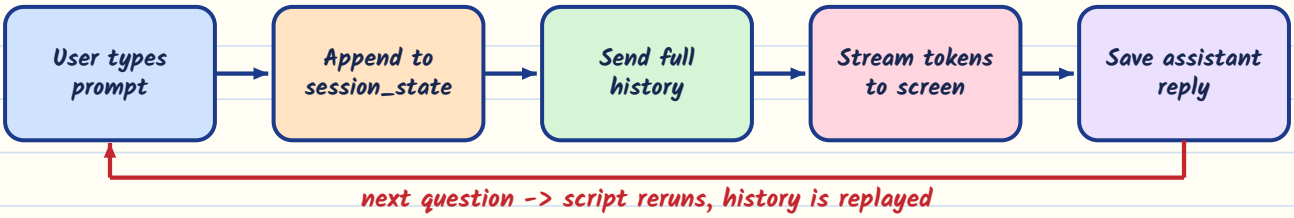
Message role	Purpose
`system`	*Sets behaviour / persona: 'You are a polite railway helpdesk bot'*
`user`	*What the person typed*
`assistant`	*The model's earlier replies (we store and resend them)*

> **REMEMBER:** *LLMs are **stateless**: the server remembers nothing between calls. The chatbot's 'memory' is simply the list of previous messages that we send again every time.*

5.4 Streamlit chat building blocks

Element	Job
`st.chat_message('user' | 'assistant')`	*Message bubble with avatar - use as `with` block*

Element	Job
st.chat_input('Ask...')	Input box pinned to the bottom; returns text on Enter, else None
st.write_stream(generator)	Prints streamed text live and returns the full string
st.session_state.messages	Our conversation memory (list of dicts)



5.5 The complete chatbot

>> chat_app.py

```
# chat_app.py  -  local chatbot: Streamlit + Ollama + Qwen
import ollama
import streamlit as st

st.set_page_config(page_title="Qwen Chat", page_icon=":material/smart_toy:")
DEFAULT_MODEL = "qwen3.5:9b"          # change to any tag from `ollama list`

# ---------- sidebar settings ----------
with st.sidebar:
    st.header("Settings")
    try:
        installed = [m.model for m in ollama.list().models]
    except Exception:
        installed = []
        st.error("Ollama is not running. Start it with: ollama serve")
    model = st.selectbox("Model", installed or [DEFAULT_MODEL])
    system_prompt = st.text_area("System prompt",
                                 "You are a helpful, concise assistant.")
    temperature = st.slider("Temperature", 0.0, 1.5, 0.7, 0.1)
    think = st.toggle("Show thinking (reasoning models)", value=False)
    if st.button("Clear chat", width="stretch"):
        st.session_state.messages = []
        st.rerun()

st.title("Chat with Qwen (local)")

# ---------- memory: list of {"role","content"} ----------
if "messages" not in st.session_state:
    st.session_state.messages = []

for m in st.session_state.messages:                  # replay history every rerun
    with st.chat_message(m["role"]):
        st.markdown(m["content"])

def stream_reply(history):
    """generator -> st.write_stream prints tokens as they arrive"""
    msgs = [{"role": "system", "content": system_prompt}] + history
    stream = ollama.chat(model=model, messages=msgs, stream=True, think=think,
                         options={"temperature": temperature})
    for chunk in stream:
        if chunk.message.thinking:
            continue                                  # skip reasoning tokens
        if chunk.message.content:
            yield chunk.message.content
```

```
if prompt := st.chat_input("Ask me anything..."):
    st.session_state.messages.append({"role": "user", "content": prompt})
    with st.chat_message("user"):
        st.markdown(prompt)
    with st.chat_message("assistant"):
        try:
            reply = st.write_stream(stream_reply(st.session_state.messages))
            st.session_state.messages.append({"role": "assistant", "content": reply})
        except Exception as e:
            st.error(f"Model error: {e}")
```

Run it with `streamlit run chat_app.py` (Ollama must be running). I tested the app's logic with Streamlit AppTest using a mocked Ollama server (message history, system prompt and streaming all worked); with a real model the same code streams real tokens.

How it works - line by line ideas

- ***Sidebar*** *lists installed models from* `ollama.list()` *- if the server is down you get a clear error instead of a crash.*
- `temperature` *(0 = focused, 1+ = creative) and* `system_prompt` *change the bot's behaviour live.*
- *The* `for m in messages` *loop* ***replays history*** *on each rerun - Streamlit forgets the screen, session_state remembers the data.*
- `stream_reply` *is a* ***generator****: it* `yield`*s text pieces;* `st.write_stream` *shows them and returns the final text.*
- `think=True` *asks reasoning models for a separate thinking trace; we skip those tokens here and show only the final answer.*
- `if prompt := st.chat_input(...)` *- the walrus operator assigns and tests in one line.*

> ***CAREFUL:*** *If you turn the thinking toggle on with a model that does not support thinking, Ollama returns an error - the* `try/except` *shows it. Leave the toggle off for such models.*

5.6 Upgrades you can add

>> small upgrades

```
# a) keep only the last N messages so the prompt never overflows the context window
history = st.session_state.messages[-20:]
msgs = [{"role": "system", "content": system_prompt}] + history

# b) persona picker in the sidebar
persona = st.selectbox("Persona", ["Study buddy", "Railway helpdesk", "Code reviewer"])
system_prompt = {"Study buddy": "Explain simply with examples.",
                 "Railway helpdesk": "You answer Indian Railways booking questions politely.",
                 "Code reviewer": "Review Python code and suggest improvements."}[persona]

# c) download the conversation
import json
st.sidebar.download_button("Export chat", json.dumps(st.session_state.messages, indent=2),
                           file_name="chat.json", mime="application/json")
```

>> show thinking separately

```
# d) show the reasoning in a collapsible box, answer streams below
with st.chat_message("assistant"):
    think_box = st.expander("Thinking", expanded=False).empty()
    answer_box = st.empty()
```

```
    thoughts, answer = "", ""
    for chunk in ollama.chat(model=model, messages=msgs, stream=True, think=True):
        if chunk.message.thinking:
            thoughts += chunk.message.thinking
            think_box.markdown(thoughts)
        if chunk.message.content:
            answer += chunk.message.content
            answer_box.markdown(answer)
st.session_state.messages.append({"role": "assistant", "content": answer})
```

Chat with your own document (mini RAG)

*RAG = "Retrieval Augmented Generation": split the document into chunks, turn chunks into **embeddings** (numbers that capture meaning), find the chunks closest to the question, and paste only those into the prompt.*

>> rag_snippet.py

```
# Tiny RAG: answer from an uploaded .txt using Ollama embeddings
import numpy as np, ollama

EMBED = "nomic-embed-text"            # ollama pull nomic-embed-text

def chunk(text, size=800):
    return [text[i:i + size] for i in range(0, len(text), size)]

def embed(texts):
    return np.array(ollama.embed(model=EMBED, input=texts).embeddings)

def top_chunks(question, chunks, vecs, k=3):
    q = embed([question])[0]
    sims = vecs @ q / (np.linalg.norm(vecs, axis=1) * np.linalg.norm(q))
    return [chunks[i] for i in sims.argsort()[::-1][:k]]

def build_prompt(question, context_chunks):
    ctx = "\n---\n".join(context_chunks)
    return f"Answer using ONLY this context:\n{ctx}\n\nQuestion: {question}"
```

>> rag_in_streamlit.py

```
# using it inside Streamlit
up = st.file_uploader("Upload a .txt file", type="txt")
if up:
    @st.cache_data
    def index(text):
        ch = chunk(text); return ch, embed(ch)
    chunks, vecs = index(up.read().decode("utf-8"))
    q = st.chat_input("Ask about the document")
    if q:
        prompt = build_prompt(q, top_chunks(q, chunks, vecs))
        with st.chat_message("assistant"):
            st.write_stream(c.message.content for c in
                ollama.chat(model="qwen3.5:9b", stream=True,
                            messages=[{"role": "user", "content": prompt}]))
```

> **TIP:** *Pull an embedding model first: `ollama pull nomic-embed-text`. For PDFs, extract text with `pypdf` first. Bigger projects use a vector store (Chroma, FAISS).*

5.7 Troubleshooting

Problem	Fix
Connection refused / 'Ollama is not running'	Start the Ollama app or run `ollama serve`; check http://localhost:11434 in a browser (shows 'Ollama is running')
model 'x' not found	`ollama pull x` or select a model from `ollama list`
Very slow replies	Use a smaller model, close other apps, check `ollama ps` for GPU use; first message is slow because the model is loading
Out of memory / crash	Smaller or more-quantised model; lower `num_ctx`
'does not support thinking'	Turn the thinking toggle off or use a thinking-capable model
Answers forget earlier turns	You are not resending history - send the whole `messages` list
Streamlit app on another machine can't connect	Set `OLLAMA_HOST` or use `ollama.Client(host='http://server:11434')`

5.8 Deploying the chatbot

Streamlit Community Cloud **cannot run Ollama** (no GPU, little RAM), so run both on a machine you control - a PC, a VM, or a GPU server - using Docker Compose. The Ollama Python client reads the `OLLAMA_HOST` environment variable automatically.

>> docker-compose.yml

```
# docker-compose.yml
services:
  ollama:
    image: ollama/ollama
    volumes:
      - ollama:/root/.ollama          # keeps downloaded models
    # (add GPU settings if you have an NVIDIA GPU + container toolkit)
  app:
    build: .                          # Dockerfile from chapter 4 (runs chat_app.py)
    ports:
      - "8501:8501"
    environment:
      - OLLAMA_HOST=http://ollama:11434
    depends_on:
      - ollama
volumes:
  ollama:
```

>> start

```
docker compose up -d --build
docker compose exec ollama ollama pull qwen3.5:9b     # download the model once
# open http://localhost:8501
```

- Do **not** expose port 11434 to the internet - the Ollama API has no login. Only the Streamlit port should be public (behind HTTPS and, ideally, authentication).
- Alternative: keep the Streamlit app on the cloud and call a model server you host elsewhere with `ollama.Client(host=...)`, or use a hosted LLM API instead of a local model.

- Each concurrent user shares the same GPU/CPU - expect slower replies when many people chat at once.

5.9 Project ideas

- **Railway helpdesk bot** - combine chapter 3 + 5: let the bot answer PNR / availability questions by calling `services.pnr_status()` and `services.availability()` (tool calling) and explain rules in simple Hindi or English.
- **Study buddy** - upload class notes, ask questions (RAG), generate quizzes with `st.radio`.
- **Code explainer** - paste code, get line-by-line explanation, export as Markdown.
- **Multi-chat app** - store chats in SQLite, list them in the sidebar like ChatGPT.
- **Vision bot** - use a vision-capable Qwen model and `st.file_uploader` / `st.camera_input` to ask questions about pictures.

6 Revision Pages

one-page cheat sheet, interview questions, practice tasks

6.1 Streamlit in one page

>> cheat sheet

```
import streamlit as st, pandas as pd

st.set_page_config(page_title="Demo", layout="wide")      # 1st st.* call
st.title("Title"); st.write("anything"); st.markdown("**md**")

name = st.text_input("Name"); n = st.slider("n", 0, 10, 5)  # widgets return values
if st.button("Go", type="primary"): st.success("clicked")     # True for one rerun

st.session_state.setdefault("count", 0)                       # memory per user session
st.session_state.count += 1

@st.cache_data(ttl=600)                                       # cache data / results
def load(): return pd.read_csv("data.csv")
@st.cache_resource                                            # cache models / connections
def model(): ...

with st.sidebar: st.selectbox("Pick", ["a", "b"])             # layout
c1, c2 = st.columns(2); c1.metric("KPI", 10, "+2")
tab1, tab2 = st.tabs(["A", "B"])
with st.expander("More"): st.write("hidden")
with st.form("f"):                                            # batch inputs
    x = st.text_input("x"); ok = st.form_submit_button("Send")

st.dataframe(df, width="stretch"); st.line_chart(df); st.image("a.png")
with st.chat_message("assistant"): st.write_stream(generator)  # chat UI
st.rerun(); st.stop()                                         # flow control
```

6.2 Mistakes beginners make

Mistake	Better
Normal variables to remember data	Use st.session_state (or a database)
Loading data/model at the top without cache	@st.cache_data / @st.cache_resource
use_container_width=True	width="stretch"
Same widget label twice	Give each a unique key=
Training the model inside the app on every rerun	Train offline, load a saved file
Business logic mixed with UI	Move logic to separate modules
Secrets in the code / GitHub	.streamlit/secrets.toml + .gitignore
Using st.button for values that must persist	Store a flag in session_state
Assuming buttons inside forms work like normal	Use st.form_submit_button

6.3 Interview questions

Q. How does Streamlit run a script?

A. It reruns the whole script top to bottom on every interaction; widgets return their current value during the run.

Q. session_state vs cache?

A. session_state stores per-user values between reruns. cache_data/cache_resource store results of expensive functions (shared across users, keyed by arguments).

Q. cache_data vs cache_resource?

A. cache_data returns a copy of serialisable data; cache_resource returns the same shared object (models, DB connections).

Q. Why use st.form?

A. To batch many inputs and avoid a rerun on every change until the submit button is pressed.

Q. How do you build a multipage app?

A. pages/ folder, or st.navigation with st.Page objects; session_state is shared across pages.

Q. How do you deploy a Streamlit app?

A. Community Cloud (GitHub + requirements.txt), Docker on a VM/Cloud Run/etc., Hugging Face Spaces. WebSockets must be allowed.

Q. How do you keep a model from reloading?

A. Load it inside a function decorated with @st.cache_resource.

Q. How does a chatbot remember context?

A. We store messages in session_state and resend the full list to the LLM each turn; the LLM itself is stateless.

Q. What is a fragment?

A. A function decorated with @st.fragment that reruns alone when its own widgets change, optionally on a timer.

6.4 Practice tasks

- **Level 1:** BMI calculator with sliders, a colour-coded result and a chart of BMI categories.
- **Level 1:** To-do list with add / complete / delete using session_state.
- **Level 2:** CSV explorer - upload any CSV, show summary stats, choose columns, draw charts, download filtered data.
- **Level 2:** Railway project upgrade - add `train_route` stops and search by any two stations on a route.
- **Level 3:** Train your own classifier (e.g. house price / spam / diabetes) and deploy it on Community Cloud.
- **Level 3:** Qwen chatbot that stores conversations in SQLite and lets you switch between saved chats.

- **Level 4:** Railway helpdesk bot - Qwen answers questions using real data from your SQLite database (tool calling).

> **TIP:** Learning path: finish the basics -> rebuild the railway app from memory -> deploy one ML app -> add an LLM feature. Official docs: **docs.streamlit.io** and the 30 Days of Streamlit challenge. Keep coding, keep rerunning!